

NEXLA

Technical Take-Home Assignment

Software Engineer

Role	Software Engineer
Time Limit	We estimate this will take about 3-4 hours.
Deliverable	Public GitHub repository URL
Documents Provided	4–5 PDF files (data)

1. About Nexla

Nexla is the leading integration platform built with AI, for AI. It takes a metadata-driven approach to converge diverse integrations across Data, Documents, Agents, Applications, and APIs into a single unified design pattern.

Nexla accelerates the development of solutions for GenAI, Analytics, and inter-company data exchange — making data users and developers up to 10x more productive through a true blend of no-code, low-code, and pro-code interfaces.

Trusted by leading companies including DoorDash, LinkedIn, Johnson & Johnson, and LiveRamp for mission-critical data workloads, Nexla is recognized in the Gartner Magic Quadrant for Data Integration Tools and is top-rated by customers on Gartner Peer Insights.

As a Software Engineer at Nexla, your job is to embed directly with strategic customers, understand their real-world data challenges, and ship production-grade solutions on top of the Nexla platform. This assignment is designed to reflect exactly that: given a messy, real-world problem — build something that works, scales, and is well-reasoned.

2. Assignment Context

One of the most common challenges Nexla's enterprise customers face is making sense of large volumes of unstructured document data — reports, contracts, whitepapers, research papers, and more. These documents often contain valuable structured and semi-structured information that is difficult to query, search, or analyze programmatically.

The Model Context Protocol (MCP) is an emerging open standard that enables AI models and agents to interact with external systems through a well-defined tool interface. Building an MCP server that exposes document intelligence capabilities is a meaningful, real-world engineering problem — the kind you will encounter regularly in this role.

In this assignment, you are provided with 4–5 PDF documents. Your task is to build an MCP server that allows an AI agent (e.g., Claude, ChatGPT, or any MCP-compatible client) to ask natural language questions and receive grounded answers derived from the content of those documents.

3. The Assignment

3.1 Core Task

Build a working MCP server that exposes a Q&A tool over the provided PDF documents. An AI agent connecting to your MCP server should be able to pose natural language questions, and your server should return accurate, grounded answers drawn from the document content.

3.2 Functional Requirements

Your MCP server must satisfy the following:

- **Document Ingestion:** The server must parse and index all 4–5 provided PDFs at startup or on demand.
- **Q&A Tool:** Expose at least one MCP tool (e.g., `query_documents`) that accepts a natural language question and returns a grounded answer with a reference to the source document and relevant section.
- **Multi-document Awareness:** The system must be able to answer questions that may require pulling context from more than one document.
- **Source Attribution:** Every answer must include the document name and, where possible, the page or section from which the answer was derived.
- **MCP Protocol Compliance:** The server must be a valid MCP server, exposing tools via the standard MCP tool-calling interface. It must be runnable locally.

3.3 Vibe Coding Setup

We want to understand how you set up and use AI-assisted development tools in your workflow. You are expected to use a vibe coding setup — meaning you should leverage an AI coding assistant (e.g., Cursor, GitHub Copilot, Claude Code, Windsurf, or similar) as part of your development process.

In your README, include a short section describing:

- Which AI coding tool(s) you used and how
- How you prompted or directed the AI — what worked, what did not
- Where you leaned on the AI vs. where you overrode or corrected it
- Your overall view on how AI tooling fits into a Software engineering workflow

There are no wrong answers here. We are evaluating your self-awareness and intentionality with these tools — not whether you used them maximally.

4. Technical Guidelines

4.1 Technology Choices

You have full freedom to choose your stack. Below are non-prescriptive suggestions to help you get started:

Component	Suggested Options (not prescriptive)
MCP Framework	Anthropic MCP SDK (Python), FastMCP
PDF Parsing	PyMuPDF, pdfplumber, pypdf, LlamaParse
Embeddings	OpenAI text-embedding-3-small, Cohere, sentence-transformers
Vector Store	ChromaDB, FAISS, Pinecone, Qdrant, Weaviate
LLM for Q&A	OpenAI GPT-5, Anthropic Claude, Ollama (local)
Language	Python

4.2 What We Are Not Looking For

- A perfect, production-deployed system — a well-structured local implementation is fine
- Any specific technology — use what you know best and can justify
- Fully productionised CI/CD or cloud deployment — local runnable is sufficient

5. Deliverables

Submit a single public GitHub repository URL. The repository must contain the following:

5.1 Working MCP Server Code

1. All source code required to run the MCP server locally
2. A working implementation of the `query_documents` tool (or equivalently named)
3. Any ingestion scripts or preprocessing steps needed to index the provided PDFs

5.2 README.md (Required — will be read carefully)

Your README is as important as your code. It must include:

- **Setup Instructions:** Step-by-step instructions to install dependencies and run the server from scratch
- **Architecture Overview:** A written or diagrammed explanation of how your system is structured
- **Tool Documentation:** Description of each MCP tool you expose — inputs, outputs, and example queries
- **Vibe Coding Section:** Your AI tooling setup and experience (see Section 3.4)

5.3 Example Interaction Log

Include at least 3 sample questions and the answers your system returned, either as a Markdown file or embedded in the README. Show that the system works and attributes answers to source documents.

6. Evaluation Criteria

Your submission will be reviewed across four dimensions:

Criterion	Weight	What We Look For
Code Quality & Architecture	35%	Clean, readable, and well-structured code. Sensible separation of concerns. Thoughtful design choices with clear intent.

MCP Protocol Understanding	25%	Correct implementation of the MCP server interface. Properly defined tools with clear schemas. A demonstrably working Q&A interaction.
Vibe Coding Setup	40%	Evidence of intentional AI-assisted development. Honest reflection on what tools were used, how, and what was learned. Shows your AI engineering mindset.

7. Submission Instructions

4. **Public** before submitting
5. **Reply to the email** from which you received this assignment with your GitHub URL
6. **[Software Engineer-AI Take-Home] Your Name**
7. We estimate this will take about 3-4 hours. We respect your time, so please stick close to that.

If you have any clarifying questions about the assignment itself, you may reply to the same email thread. We will do our best to respond promptly, but the clock does not pause during that time.

8. A Note from the Team

We are not looking for perfection — we are looking for how you think. A clean, well-documented solution with honest trade-offs and a thoughtful README will score higher than a technically sophisticated system with no explanation of why choices were made.

This role operates like a technical co-founder for each customer engagement. We want to see that you can own an outcome, reason under ambiguity, and communicate your decisions clearly. That is exactly what this assignment is designed to surface.

Good luck — and we look forward to seeing what you build.